Node.js & Asynchronous JavaScript

INSTRUCTOR NAME: SANTHANALAKSHMI

INSTITUTION NAME: CHRIST UNIVERSITY

CLASS : 5BTCSIOT

DATE OF SUBMISSION: 27/08/2026

Practical Question Paper - Set 2

NAME	REGISTRATION NUMBER	CHRIST EMAIL ID
BARATH C G	2462507	barath.cg@btech.christuniversity.in

Department of Computer Science and Engineering

School of Engineering and Technology

CHRIST (Deemed to be University)

Kumbalgodu, Bangalore - 560 074

August-2026

27/8/26

Node.js & Asynchronous JavaScript

BARATH C G
2462507
5BTCSIOT

Theory Question Paper - Set B

Q1). Node.js will show an error like "Error: Cannot find module '.../app.js'". The developer should move to correct folder containing app.js or provide correct file path.

Q2). V8 executes JavaScript code, libuv handles asynchronous operations such as network requests, event loop & file operations. Together, they allow Node.js to execute JavaScript while handling I/O asynchronously.

Q3). The fs documentation provides method syntax, parameters, return values, and examples. It helps developers chose between synchronous methods like readFileSync() and asynchronous methods like readFile()

Q5).
```
[
  'path/to/node',
  'path/to/app.js',
  'hello',
  'world'
]
```
path & script path followed by command line argument

The first two value are Node.js executable -

Q7). getUser() returns a Promise, so user is not the actual user object. Use await to get resolved value.

```
Const user = await getUser();
Console.log(user.name);
```

Q8).
```
Const fs = require('fs');
function readFileCallback(path, callback) {
  fs.readFile(path, 'utf8', (err, data) => {
  if (err) return callback(err, null);
```

```
    callback(null, data);
  });
}
```

Q10). Function, function delay(ms){
```
    return new Promise((resolve) => {
      setTimeout(resolve, ms);
    });
}
```

Q11).
```
async function f(){
  try{
    await Promise.reject('err1');
  }catch(e){
    console.log('caught:', e);
  }
}
```

Answer:
caught: err1

Q12).
```
function a(){
  console.log('a');
  b();
}
function b(){
  console.log('b');
}
a();
console.log('c');
```

Answer:
a
b
c

Q15). math.js
```
function add(a,b){
  return a+b;
}
module.exports = add;
```

app.js
```
const add = require('./math')
console.log(add(2,3));
```

Countdown Timer & Notification App

Node.js & Asynchronous JavaScript CIA-2 Practical Answers - 10 Tasks

Topic	Countdown Timer & Notification App
Tasks Covered	Tasks 1, 2, 3, 4, 5, 8, 9, 10, 11 and 13
Language Used	JavaScript with Node.js

Task 1: Countdown App Setup and First Run

Question: Initialize a new Node.js project using npm init -y and create an entry file countdown.js. Run countdown.js using node countdown.js and confirm it prints "Countdown App Ready".

Code Screenshot

task01_countdown.js — JavaScript

```
1  // countdown.js
2  // Project setup command: npm init -y
3
4  console.log("Countdown App Ready");
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> npm init -y
Wrote to countdown-app\package.json
PS> node countdown.js
Countdown App Ready
```

Summary: Initialized the project workflow and created the countdown entry file. The successful run confirms that Node.js executes the file and prints the required readiness message.

Used: npm init -y, Node.js command line, console.log()

Task 2: Node.js Architecture: V8 and libuv

Question: Add a comment explaining how V8 and libuv let countdown.js keep accepting terminal input while a timer runs in the background. Demonstrate this by starting a setTimeout and immediately printing another message before the timer fires.

Code Screenshot

task02_architecture.js — JavaScript

```javascript
// V8 executes the JavaScript on the main thread.
// libuv manages the timer in the background and queues its callback.
// Therefore, Node.js can keep handling terminal I/O while the timer waits.

console.log("Starting background timer...");

setTimeout(() => {
  console.log("Timer callback executed.");
}, 500);

console.log("This message prints before the timer fires.");
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task02_architecture.js
Starting background timer...
This message prints before the timer fires.
Timer callback executed.
```

Summary: V8 executes the JavaScript while libuv manages the scheduled timer and later queues its callback. The immediate message appears before the timer callback, demonstrating non-blocking timer behavior.

Used: V8, libuv, setTimeout(), event loop

Task 3: Timers Documentation and Basic Countdown

Question: Refer to the official Node.js documentation for the timers module and list, as a comment, the method names used. Adapt an example to build a basic countdown using setInterval.

Code Screenshot

task03_basic_countdown.js — JavaScript

```javascript
// Official Node.js timers methods used:
// setInterval() and clearInterval()

let secondsRemaining = 3;
console.log(`Countdown starts at ${secondsRemaining} seconds`);

const intervalId = setInterval(() => {
  secondsRemaining -= 1;
  console.log(`${secondsRemaining} second(s) remaining`);

  if (secondsRemaining === 0) {
    clearInterval(intervalId);
    console.log("Countdown complete!");
  }
}, 1000);
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task03_basic_countdown.js
Countdown starts at 3 seconds
2 second(s) remaining
1 second(s) remaining
0 second(s) remaining
Countdown complete!
```

Summary: Used setInterval() to update the remaining time every second and clearInterval() to stop exactly at zero. The method names are documented in comments as required.

Used: setInterval(), clearInterval()

Official reference: Node.js Timers documentation

Task 4: Node REPL and Date Difference

Question: In the Node REPL, test a snippet that calculates the seconds remaining between two Date objects. Move the working snippet into countdown.js.

Code Screenshot

task04_date_difference.js — JavaScript

```javascript
// REPL-tested expression:
// (endTime - startTime) / 1000

const startTime = new Date("2026-08-27T10:00:00Z");
const endTime = new Date("2026-08-27T10:01:30Z");

const secondsRemaining = (endTime - startTime) / 1000;

console.log(`Start time: ${startTime.toISOString()}`);
console.log(`End time:   ${endTime.toISOString()}`);
console.log(`Seconds remaining: ${secondsRemaining}`);
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task04_date_difference.js
Start time: 2026-08-27T10:00:00.000Z
End time:   2026-08-27T10:01:30.000Z
Seconds remaining: 90
```

Summary: Subtracted two Date objects and divided the millisecond difference by 1000. The same REPL-tested expression reports 90 seconds when executed from the file.

Used: Node REPL, Date objects, arithmetic conversion

Task 5: Command-Line Duration and Cancel Input

Question: Modify countdown.js to accept the countdown duration in seconds through process.argv. Use process.stdin so the user can type "cancel" to stop the countdown early.

Code Screenshot

task05_cli_cancel.js — JavaScript

```javascript
const duration = Number(process.argv[2]);

if (!Number.isFinite(duration) || duration <= 0) {
  console.error("Enter a positive duration in seconds.");
  process.exitCode = 1;
} else {
  let remaining = duration;
  console.log(`Countdown started for ${duration} seconds.`);
  console.log('Type "cancel" and press Enter to stop it.');

  const intervalId = setInterval(() => {
    remaining -= 1;
    console.log(`${remaining} second(s) remaining`);

    if (remaining === 0) {
      clearInterval(intervalId);
      console.log("Countdown complete!");
      process.stdin.pause();
    }
  }, 1000);

  process.stdin.setEncoding("utf8");
  process.stdin.on("data", (input) => {
    if (input.trim().toLowerCase() === "cancel") {
      clearInterval(intervalId);
      console.log("Countdown cancelled early.");
      process.stdin.pause();
    }
  });
}
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> "cancel" | node task05_cli_cancel.js 5
Countdown started for 5 seconds.
Type "cancel" and press Enter to stop it.
Countdown cancelled early.
```

Summary: Read the duration from process.argv and listened for terminal data through process.stdin. Entering cancel clears the interval and stops the countdown before zero.

Used: process.argv, process.stdin, clearInterval()

Task 8: Callback-Based Asynchronous Function

Question: Write checkTimeLeftCallback(seconds, callback), simulate a delay with setTimeout, then call it and log the remaining time inside the callback.

Code Screenshot

task08_callback.js — JavaScript

```javascript
function checkTimeLeftCallback(seconds, callback) {
  // Simulate an asynchronous operation before returning the value.
  setTimeout(() => {
    if (seconds < 0) {
      callback(new Error("Seconds cannot be negative"), null);
      return;
    }

    callback(null, seconds);
  }, 300);
}

checkTimeLeftCallback(5, (error, timeLeft) => {
  if (error) {
    console.error(error.message);
    return;
  }

  console.log(`Time left: ${timeLeft} seconds`);
});
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task08_callback.js
Time left: 5 seconds
```

Summary: Created an error-first callback function that waits asynchronously before returning the remaining seconds. The callback handles either an error or the successful time value.

Used: callback function, setTimeout(), error-first callback pattern

Task 9: Countdown with Node Timers

Question: Use setInterval to print the remaining seconds every second and stop it with clearInterval exactly at zero. Use a separate setTimeout to trigger a "Time's up!" notification after the countdown ends.

Code Screenshot

task09_timers.js — JavaScript

```javascript
let remaining = 3;
console.log(`Remaining: ${remaining} seconds`);

const intervalId = setInterval(() => {
  remaining -= 1;
  console.log(`Remaining: ${remaining} seconds`);

  if (remaining === 0) {
    clearInterval(intervalId);
    console.log("Interval stopped at zero.");
  }
}, 1000);

setTimeout(() => {
  console.log("Time's up!");
}, remaining * 1000 + 50);
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task09_timers.js
Remaining: 3 seconds
Remaining: 2 seconds
Remaining: 1 seconds
Remaining: 0 seconds
Interval stopped at zero.
Time's up!
```

Summary: The interval prints each remaining value and is cleared at zero. A separate timeout displays the final notification immediately after the countdown finishes.

Used: setInterval(), clearInterval(), setTimeout()

Task 10: Promise-Based Time Check

Question: Rewrite checkTimeLeftCallback as checkTimeLeftPromise(seconds). Chain .then() and .catch() to log the result or an error.

Code Screenshot

task10_promise.js — JavaScript

```javascript
function checkTimeLeftPromise(seconds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!Number.isFinite(seconds) || seconds < 0) {
        reject(new Error("Invalid countdown duration"));
        return;
      }

      resolve(seconds);
    }, 300);
  });
}

checkTimeLeftPromise(8)
  .then((timeLeft) => {
    console.log(`Promise resolved: ${timeLeft} seconds remaining`);
  })
  .catch((error) => {
    console.error(`Promise rejected: ${error.message}`);
  });
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task10_promise.js
Promise resolved: 8 seconds remaining
```

Summary: Wrapped the delayed check in a Promise that resolves valid durations and rejects invalid values. The result is handled using a .then()/.catch() chain.

Used: Promise, resolve(), reject(), .then(), .catch()

Task 11: Async-Await with Try/Catch

Question: Write runCountdownAsync(seconds), await checkTimeLeftPromise(seconds) inside try/catch, and demonstrate the catch block by passing a negative duration.

Code Screenshot

task11_async_await.js — JavaScript

```javascript
function checkTimeLeftPromise(seconds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!Number.isFinite(seconds) || seconds < 0) {
        reject(new Error("Duration must be zero or greater"));
        return;
      }
      resolve(seconds);
    }, 300);
  });
}

async function runCountdownAsync(seconds) {
  try {
    const timeLeft = await checkTimeLeftPromise(seconds);
    console.log(`Async result: ${timeLeft} seconds remaining`);
  } catch (error) {
    console.log(`Caught error: ${error.message}`);
  }
}

runCountdownAsync(-2);
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task11_async_await.js
Caught error: Duration must be zero or greater
```

Summary: Awaited the Promise inside a protected try block. Passing -2 rejects the Promise, so the catch block prints the validation error.

Used: async, await, try/catch, Promise rejection

Task 13: Event Loop and Job Queue Order

Question: Combine setTimeout, a resolved Promise .then(), and a synchronous console.log. Predict the output order in a comment, run it, and confirm whether the prediction was correct.

Code Screenshot

task13_event_loop.js — JavaScript

```javascript
// Predicted order:
// 1. Synchronous message
// 2. Promise microtask
// 3. setTimeout callback

setTimeout(() => {
  console.log("3. setTimeout callback");
}, 0);

Promise.resolve().then(() => {
  console.log("2. Promise microtask");
});

console.log("1. Synchronous message");

// The observed output confirms the prediction because the Promise job
// queue is processed before the timer callback queue.
```

Output Screenshot

Terminal - Node.js — PowerShell

```
PS> node task13_event_loop.js
1. Synchronous message
2. Promise microtask
3. setTimeout callback
```

Summary: The synchronous log runs first, followed by the Promise microtask and then the timer callback. The observed output matches the prediction written in the code comments.

Used: call stack, Promise job queue, timer callback queue, event loop

GITHUB link: [BarathCG/L-T-SEM-5 repository]